COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE-010 Data Structures and Algorithm | **SECTION:** | CPE21S4 |
| **DATE PERFORMED:** | 10/21/26 | **DATE SUBMITTED:** | 10/21/26 |
| **NAME:** | Pangyarihan,Jenny Rose<br>Peñaflorida,Wrio Carlo | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 14.1 Algorithmic Complexity

### 1. PROCEDURE OUTPUT

Tabulated below, are a number of functions against n (from 1 to 10)
A = (log2 n) {log to base 2 of n}
B = n {linear in n}
C = (* n (log2 n)) {n log n}
D = (* n n) {quadratic in n}
E = (* n n n) {cubic in n}
F = (expt 2 n) {exponential in n}
G = (expt 3 n) {exponential in n}
H = (fact n) {factorial in n}

| n | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 | 0.0 | 1 | 0.0 | 1 | 1 | 2 | 3 | 1 |
| 2 | 1.0 | 2 | 2.0 | 4 | 8 | 4 | 9 | 2 |
| 3 | 1.0 | 3 | 4.0 | 9 | 27 | 8 | 27 | 6 |
| 4 | 2.0 | 4 | 8.0 | 16 | 64 | 16 | 81 | 24 |
| 5 | 2.0 | 5 | 11.0 | 25 | 125 | 32 | 243 | 120 |
| 6 | 2.0 | 6 | 15.0 | 36 | 216 | 64 | 729 | 720 |
| 7 | 2.0 | 7 | 19.0 | 49 | 343 | 128 | 2187 | 5040 |
| 8 | 3.0 | 8 | 24.0 | 64 | 512 | 256 | 6561 | 40320 |
| 9 | 3.0 | 9 | 28.0 | 81 | 729 | 512 | 19683 | 362880 |
| 10 | 3.0 | 10 | 33.0 | 100 | 1000 | 1024 | 59049 | 3628800 |

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x".

| n | F | G | H |
|---|---|---|---|
| 10 | 10^-10 | 10^-8 | 10^-6 |
| 20 | 10^-7 | 10^-3 | 10^4 |
| 30 | 10^-4 | 10^0 | 10^18 |
| 40 | 10^-1 | 10^5 | 10^34 |
| 50 | 10^1 | 10^10 | 10^50 |
| 60 | 10^4 | 10^15 | 10^68 |
| 70 | 10^7 | 10^19 | 10^86 |
| 80 | 10^10 | 10^24 | 10^105 |
| 90 | 10^13 | 10^29 | 10^124 |
| 100 | 10^16 | 10^34 | 10^144 |

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take
10,000,000,000,000,000,000,000,000,000,000,000 years
approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, ie 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say, that from a computational perspective, the universe is a small thing.
Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

| | **Best Case** | **Worst Case** | **Analysis** |
|---|---|---|---|
| **Algorithm 1**<br>**Bubble Sort** | Already sorted input: one pass, no swaps, so n − 1 comparisons. O(n) | Reverse-sorted input: n(n − 1)/2 comparisons. O(n2) | Space is O(1) because it sorts in place. It is simple and stable but the quadratic worst case makes it unusable for large n. |
| **Algorithm 2**<br>**Merge Sort** | Sorted input still splits and merges, about (n/2) log2 n comparisons. O(n log n) | About n log2 n − n + 1 comparisons. O(n log n) | Divide-and-conquer: the array halves log2 n times and each level merges n elements. Space is O(n) for the temporary array. It is stable and its running time barely depends on the input order. |

**Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools**

| n | Bubble Sort best | Bubble Sort worst | Merge Sort best | Merge Sort worst |
|---|---|---|---|---|
| **100** | **0.0005 ms** | **0.0797 ms** | 0.0100 ms | **0.0105 ms** |
| **1000** | **0.0058 ms** | **8.5831 ms** | 0.1338 ms | **0.1312 ms** |
| **1000** | **0.0414 ms** | **739.0350 ms** | 1.7483 ms | **1.7500 ms** |

**Table 14-2. Best- and Worst-Case Analysis using Experimental Tools**

**Analysis**

The experiment agrees with the theory. When n went up 10 times, Bubble Sort's worst case went up about 108 times (0.0797 to 8.58 ms) and then about 86 times (8.58 to 739 ms). That is close to the 100 times predicted by n2. Merge Sort went up only about 13 times each step, which matches n log n (10 times the data times a slightly bigger log factor).
At n = 10,000 the worst-case Bubble Sort took 739 ms against 1.75 ms for Merge Sort, so Merge Sort was about 420 times faster, and the gap keeps widening as n grows.
Bubble Sort has one real strength. On already sorted data it needed only 0.0414 ms for n = 10,000, faster than Merge Sort's 1.75 ms, because it does a single pass. Merge Sort's best and worst times are almost identical, which makes it predictable. The trade-off is memory: Merge Sort needs O(n) extra space and Bubble Sort needs O(1).

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.
For the theoretical analysis, the deliverable must be:
Pseudocode Analysis
Time and Space Complexities

**LinearSearch**
(A[0..n-1], x)
for i <- 0 to n - 1 do
if A[i] = x then return i
return -1

**BinarySearch**
(A[0..n-1], x)
low <- 0; high <- n - 1
while low <= high do
mid <- (low + high) / 2
if A[mid] = x then return mid
else if A[mid] < x then low <- mid + 1
else high <- mid - 1
return -1

| | Best Case | Worst Case | Space |
|---|---|---|---|
| **Linear search** | O(1), found at the first element | O(n), checks all n elements | O(1) |
| **Binary search** | O(1), found at the middle | O(log n), halves the range until empty | O(1) iterative (O(log n) if written recursively) |

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

| Input Size (N) | Elapsed Time for Binary Search (Worst-Case) | Elapsed Time for Linear Search (Worst-Case) |
|---|---|---|
| 1,000 | 0.0524 µs | 3.8461 µs |
| 10,000 | 0.0836 µs | 35.6512 µs |
| 100,000 | 0.1110 µs | 332.2943 µs |

**Analysis:**

**Peñaflorida**
Linear search grew by about 9 to 10 times each time N grew 10 times (3.85 to 35.7 to 332 µs), which is the straight-line O(n) behavior. Binary search barely moved: 0.052 to 0.084 to 0.111 µs. That is about 1.6 times and then 1.3 times, which fits log2 N going from about 10 to  13 to 17. At N = 100,000, binary search was about 3,000 times faster than linear search, and the gap was already about 73 times at N = 1,000.

**Pangyarihan**

The results match what the theory predicts. Linear search took 3.8461 µs at N = 1,000, then 35.6512 µs at 10,000, and 332.2943 µs at 100,000, so its time grew about 9 to 10 times every time the input grew 10 times. That is the steady O(n) growth expected, since it may have to check every item. Binary search stayed almost flat, going from 0.0524 µs to 0.0836 µs to 0.1110 µs, because each 10x increase in size only adds a few more steps, which fits O(log n). As a result, the gap between the two kept getting bigger: linear search was about 73 times slower at 1,000 items and nearly 3,000 times slower at 100,000. This shows that binary search is much better for large sorted data, though it only works if the data is already sorted.

**CONCLUSION:**

**Peñaflorida**
Running time depends on many things, but the algorithm itself and the input size matter most. Big-O keeps only the dominant term and ignores constants, I also learned the difference between best,worst and average case, and that intractable problems (2n, n!) cannot be rescued by faster hardware. Comparing Bubble Sort and Merge Sort showed theory and measurement lining up. The ×100 jump for n2 and the ×13 jump for n log n both appeared in my timings. Counting comparisons instead of seconds gave exact, machine-independent numbers that matched the formulas.Binary search is O(log n) and linear search is O(n), and the measurements showed it clearly: the bigger N got, the bigger the gap. The catch is that binary search needs sorted data, so the right choice depends on whether the data is sorted and how often I search it.I think I did well on this activity because I could predict the results from the theory before running the code, and the numbers matched. My areas for improvement are working out recurrences

**Pangyarihan:**

Linear search grows steadily as the input gets bigger, while binary search barely changes, so binary search is much faster on large sorted data. In the procedure, I timed both searches at their worst case using 1,000, 10,000, and 100,000 items, and the results matched the theory. Linear search took about 10 times longer each time the input grew 10 times, but binary search only went from 0.0524 µs to 0.1110 µs. Small differences from the expected results were probably caused by background processes and timer limits on the computer.

**3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT**

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

COMPUTER ENGINEERING

QUEZON CITY

**Extent of Use:**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.